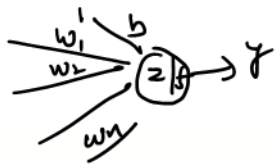
import torch.nn as nn → loss

nn.Module (All the Toolkit)

X.shape[1]= 


```python
class MySimpleNN():

  def __init__(self, X):

    self.weights = torch.rand(X.shape[1], 1, dtype=torch.float64, requires_grad=True)
    self.bias = torch.zeros(1, dtype=torch.float64, requires_grad=True)

  def forward(self, X):
    z = torch.matmul(X, self.weights) + self.bias
    y_pred = torch.sigmoid(z)
    return y_pred

  def loss_function(self, y_pred, y):
    # Clamp predictions to avoid log(0)
    epsilon = 1e-7
    y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)

    # Calculate loss
    loss = -(y_train_tensor * torch.log(y_pred) + (1 - y_train_tensor) * torch.log(1 - y_pred)).mean()
    return loss
```

① ② ③

L1 L10

update) $W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$

zeros

```python
# Hyper Parmeter
learning_rate = 0.1
epochs = 25
# create model
model = MySimpleNN(X_train_tensor)

# define loop
for epoch in range(epochs):

  # forward pass
  y_pred = model.forward(X_train_tensor)

  # loss calculate
  loss = model.loss_function(y_pred, y_train_tensor)

  # backward pass
  loss.backward()

  # parameters update
  with torch.no_grad():
    model.weights -= learning_rate * model.weights.grad
    model.bias -= learning_rate * model.bias.grad

  # zero gradients
  model.weights.grad.zero_()
  model.bias.grad.zero_()

  # print loss in each epoch
  print(f'Epoch: {epoch + 1}, Loss: {loss.item()}')
```

④

```python
class DeepModel(nn.Module):

    def __init__(self, num_features):

        super().__init__()

        self.linear1 = nn.Linear(num_features, 3)
        self.relu = nn.ReLU()

        self.linear2 = nn.Linear(3, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):

        out = self.linear1(x)
        out = self.relu(out)

        out = self.linear2(out)
        out = self.sigmoid(out)

        return out
```

(A) ① (B) 2 z f(z)

OR

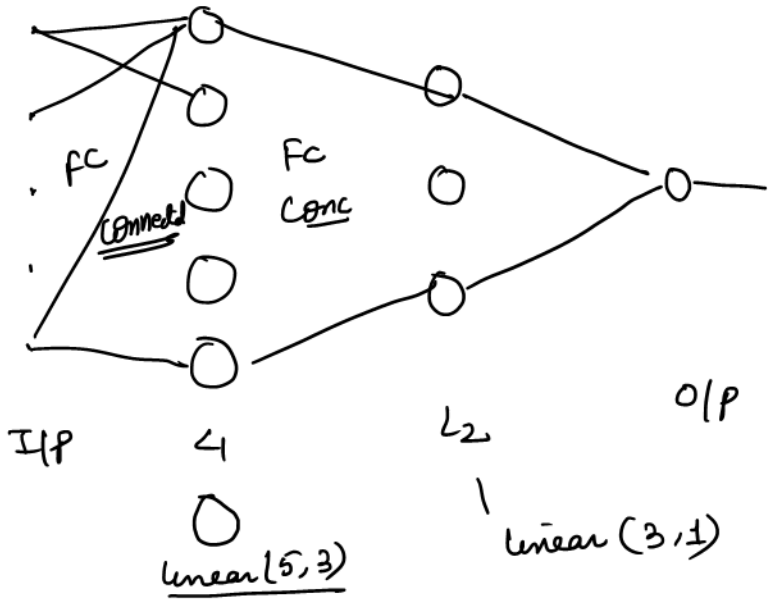
```python
class SequentialModel(nn.Module):

    def __init__(self, num_features):

        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(num_features, 3),
            nn.ReLU(),
            nn.Linear(3, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.network(x)
```

(A) (B)

③ loss_function = nn.BCELoss() ✓ (object create)

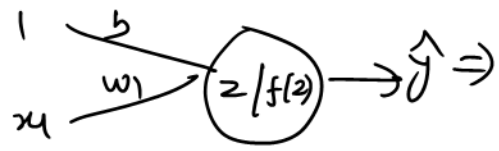
④ optimizer = torch.optim.SGD(seq_model.parameters(),lr=0.01) ✓

```python
X = torch.randn(10, 5)
y_train = torch.randint(0, 2, (10,1)).float()
seq_model = SequentialModel(5)
preds = seq_model(X)
loss = loss_function(preds, y_train)
```



Machine learning | Deep learning

features should be in number format → number



$z = w_1 x_1 + b$

$\hat{y} = \sigma(z)$

Natural Language → Hindi / Urdu etc

Text → convert — number

EngSimilar $(w_1, w_2) > \epsilon(w_1, w_3)$ w_1 good — more similar w_3 Elephant

w_2 Best

$w \rightarrow$ numeric vector

Hari is riding the bike. ✓

Bike is riding the Hari. ✓

EngSimilar $(v_1, v_2) >$ EngSim (v_1, v_3)

length $(v_1, v_2) <$ length (v_1, v_3)

eg. w_1 w_2 w_3

$\downarrow$ $\downarrow$ $\downarrow$

v_1 v_2 v_3

B

BOW, tf-idf , w2v , avg-w2v (tf-idf).w2v

eg.

r1: This pasta is very tasty and affordable.
r2: This pasta is not tasty and is affordable
r3: This pasta is delicious and cheap.
r4: Pasta is tasty and pasta tastes good.

The and is

Binary BOW

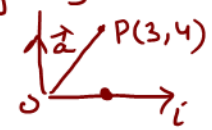
dict: {This, pasta, is, very, tasty, and, affordable, not, delicious, cheap, tastes, good}

r1: [1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]
r2=[1, 1, 2 0 1 1 1 0 0 0 0 0 0]

Counting Based BOW

$\vec{a} = 3i + 4j$

Array $\vec{a} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ OR [3, 4]



① NO semantic meaning captured.
② NO sequence info / NO grammer followed.
③ sparse vector

Text processing:

① Removal of stopword → dim ↓ → not contributing any value

② tasty Vs delicious

Root word

history > histori
historical

finally, final, finalized > final

(Rule based)

stemming

history > histori
historical

SSES → SS
caresses → caress

fast — Porter stemmer
computer — snowball stemmer

Lemmatization

history > history
historical

time
human error